CSE 5819 - Assignment 8

Riley Francis - rif17002

Recurrent Neural Networks

1.

a.

Since there are just two characters A and B, their one-hot vectors will be

$$A = \begin{bmatrix}1\\0\end{bmatrix} \quad B = \begin{bmatrix}0\\1\end{bmatrix}$$

b.

We have:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$

So, assuming $h_0 = 0$, h_1 will become:

$$h_1 = \tanh\left(\begin{bmatrix}1 & 0 & 1\\ -1 & 1 & 0\\ 0 & 2 & -1\end{bmatrix}(0) + \begin{bmatrix}1 & -1\\ 0 & 2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} + 0\right) = \tanh\left(\begin{bmatrix}1\\0\\-1\end{bmatrix}\right) \approx \begin{bmatrix}0.762\\0\\-0.762\end{bmatrix}$$

h_2 will become:

$$h_2 = \tanh\left(\begin{bmatrix}1 & 0 & 1\\ -1 & 1 & 0\\ 0 & 2 & -1\end{bmatrix}\begin{bmatrix}0.762\\0\\-0.762\end{bmatrix} + \begin{bmatrix}1 & -1\\ 0 & 2\\ -1 & 1\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} + 0\right) = \tanh\left(\begin{bmatrix}-1\\1.238\\1.762\end{bmatrix}\right) \approx \begin{bmatrix}-0.762\\0.845\\0.943\end{bmatrix}$$

h_3 will become:

$$h_3 = \tanh\left(\begin{bmatrix}1 & 0 & 1\\ -1 & 1 & 0\\ 0 & 2 & -1\end{bmatrix}\begin{bmatrix}-0.762\\0.845\\0.943\end{bmatrix} + \begin{bmatrix}1 & -1\\ 0 & 2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} + 0\right) = \tanh\left(\begin{bmatrix}1.181\\1.607\\-0.253\end{bmatrix}\right) \approx \begin{bmatrix}0.828\\0.923\\-0.247\end{bmatrix}$$

c.

We have:

$$\hat{y}_t = \text{softmax}(W_y h_t + b_y)$$

$\hat{y}_1$ will become:

$$\hat{y}_1 = \text{softmax}\left(\begin{bmatrix}1 & -1 & 0\\ 0 & 2 & -1\end{bmatrix}\begin{bmatrix}0.762\\ 0\\ -0.762\end{bmatrix} + 0\right) = \text{softmax}\left(\begin{bmatrix}0.762\\ 0.762\end{bmatrix}\right) = \begin{bmatrix}0.5\\ 0.5\end{bmatrix}$$

$\hat{y}_2$ will become:

$$\hat{y}_2 = \text{softmax}\left(\begin{bmatrix}1 & -1 & 0\\ 0 & 2 & -1\end{bmatrix}\begin{bmatrix}-0.762\\ 0.845\\ 0.943\end{bmatrix} + 0\right) = \text{softmax}\left(\begin{bmatrix}-1.607\\ 0.747\end{bmatrix}\right) = \begin{bmatrix}0.087\\ 0.913\end{bmatrix}$$

$\hat{y}_3$ will become:

$$\hat{y}_2 = \text{softmax}\left(\begin{bmatrix}1 & -1 & 0\\ 0 & 2 & -1\end{bmatrix}\begin{bmatrix}0.828\\ 0.923\\ -0.247\end{bmatrix} + 0\right) = \text{softmax}\left(\begin{bmatrix}-0.095\\ 2.093\end{bmatrix}\right) = \begin{bmatrix}0.101\\ 0.899\end{bmatrix}$$

d.

The model will predict B after $t = 3$ with 89.9% confidence.

2.

We know the following:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$
$$\hat{y}_t = \text{softmax}(W_y h_t + b_y)$$

Let $\sum_k L_k$ be the total loss over all time steps. We want to find $\frac{\partial L}{\partial h_t}$. This term can be expressed as the sum of direct path and the indirect path. So through the chain rule, we have:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \frac{\partial L}{\partial h_{t+1}}\frac{\partial h_{t+1}}{\partial h_t}$$

For the direct path term, we pass through o_t, so $\frac{\partial L_t}{\partial h_t}$ can be written as

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial o_t}\frac{\partial o_t}{\partial h_t}$$

Then, $\frac{\partial L_t}{\partial o_t} = \hat{y}_t - y_t$ and because $o_t = W_y h_t + b_y$, $\frac{\partial o_t}{\partial h_t} = W_y$, so

$$\frac{\partial L_t}{\partial h_t} = W_y^\top(\hat{y}_t - y_y)$$

Next, for the indirect path term, we pass via the hidden layer h_{t+1}. Using the chain rule, we get:

$$\frac{\partial L}{\partial h_{t+1}}\frac{\partial h_{t+1}}{\partial h_t} = \frac{\partial L}{\partial h_{t+1}}\frac{\partial h_{t+1}}{\partial a_{t+1}}\frac{\partial a_{t+1}}{\partial h_t}$$

Then, $\frac{\partial h_{t+1}}{\partial a_{t+1}} = 1 - h_{t+1}^2$ and $\frac{\partial a_{t+1}}{\partial h_t} = W_h$. When these get combined element-wise, we get

$$\frac{\partial L}{\partial h_{t+1}}\frac{\partial h_{t+1}}{\partial h_t} = W_h^\top\left(\frac{\partial L}{\partial h_{t+1}} \odot (1 - h_{t+1}^2)\right)$$

Finally, combining everything together, we get:

$$\frac{\partial L}{\partial h_t} = W_y^\top(\hat{y}_t - y_y) + W_h^\top\left(\frac{\partial L}{\partial h_{t+1}} \odot (1 - h_{t+1}^2)\right)$$

Convolution Neural Networks

1a.

With an input $I \in \mathbb{R}^{n \times n \times d_0}$ given d_0 input channels, a kernel size $k \times k$, stride s, and padding p, the size of the output is:

$$m = \left\lfloor \frac{n - k + 2p}{s} \right\rfloor + 1$$

Then the output is

$$O \in \mathbb{R}^{m \times m \times d_1}$$

given d_1 output channels.

1b.

Here we multiply the kernel matrix K with each "window" of the image matrix I component-wise. The output of each of these matrix multiplications forms the output matrix:

$$\begin{bmatrix} 5(1) + 1(0) + 3(0) + 0(-1) & 1(1) + 1(0) + 0(0) + 2(-1) \\ 3(1) + 0(0) + 4(0) + 4(-1) & 0(1) + 2(0) + 4(0) + 0(-1) \end{bmatrix} = \begin{bmatrix} 5 & -1 \\ -1 & 0 \end{bmatrix}$$

Part 1

1. What are neural networks?

Neural networks are computational models inspired by the structure of the human brain. They consist of interconnected nodes (neurons) that process information through weighted connections. By adjusting these weights, neural networks learn to approximate functions from data.

2. What is the difference between neural networks and artificial neural networks?

“Neural networks” can refer broadly to networks of biological neurons in the brain. “Artificial neural networks” (ANNs) are computational models designed to mimic some behaviors of biological networks. ANNs use mathematical operations and optimization rather than biological signals.

3. How many kinds of artificial neural networks?

There are many types, but common families include feed-forward networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformers. Each type is suited to different patterns in data such as images, sequences, or long-range dependencies. Specialized variants like autoencoders, GANs, or GNNs also exist.

4. What is a neuron?

A neuron in an ANN is a computational unit that takes inputs, multiplies them by weights, adds a bias, and applies an activation function. Mathematically, a neuron computes

$$y = \sigma(w^\top x + b).$$

It outputs a transformed signal that flows to other neurons.

5. What does an activation function do?

An activation function introduces non-linearity into the network so it can learn complex patterns. Without it, the model would reduce to a simple linear function regardless of depth. Common activation functions include ReLU, sigmoid, and tanh.

6. What is a layer in neural networks?

A layer is a group of neurons that operate at the same depth in the network. Layers transform the data from one representation to another through learned weights. Stacking layers allows increasingly abstract feature extraction.

7. What is a loss function when training a neural network?

A loss function measures how far the model's predictions deviate from the true target values. Training aims to minimize this loss by adjusting network weights. Common losses include mean squared error (MSE) and cross-entropy.

8. Why single-layer perceptron cannot approximate XOR logic function?

A single-layer perceptron can only represent linearly separable functions. XOR is not linearly separable, meaning no straight line can divide its classes. Therefore the perceptron cannot learn XOR regardless of training.

9. Why multi-layer perceptron can approximate many functions (universal approximation theorem)?

Adding hidden layers introduces non-linear transformations, allowing MLPs to build complex decision boundaries. The universal approximation theorem states that a network with at least one hidden layer and sufficient neurons can approximate any continuous function on a compact domain. This makes MLPs extremely powerful function approximators.

10. What is a recurrent neural network?

A recurrent neural network (RNN) is designed to process sequential data by maintaining a hidden state across time steps. This hidden state captures information from previous inputs. RNNs are commonly used for tasks like language modeling or time-series analysis.

11. What is a convolution neural network?

A convolutional neural network (CNN) uses convolutional filters to extract local spatial patterns from data like images. These filters slide across the input to detect features such as edges and textures. CNNs are highly effective for image recognition and computer vision tasks.

12. What does it mean backpropagation?

Backpropagation is the algorithm used to compute gradients of the loss with respect to network weights. It applies the chain rule to propagate errors backward through layers. These gradients are then used to update weights via optimization methods like SGD or Adam.

13. What does backpropagation do?

Backpropagation determines how much each weight contributed to the final error. It computes partial derivatives

$$\frac{\partial L}{\partial w}$$

for every weight in the network. This enables the optimizer to adjust weights in the direction that reduces the loss.

14. How are the weights of a neural network determined or optimized?

Weights start from an initialization (random or heuristic) and are iteratively updated during training. An optimizer like SGD or Adam uses gradients from backpropagation to adjust weights. Over many epochs, this process minimizes the loss function.

15. What is the problem of gradient vanishing and gradient explosion?

In deep networks, gradients can shrink (vanish) or grow uncontrollably (explode) as they propagate backward. Vanishing gradients cause very slow learning in early layers, while exploding gradients lead to unstable training. These issues motivated architectures like LSTMs, ResNets, and normalization techniques.

16. Why might ReLU be preferred in very deep networks?

ReLU avoids saturation for positive inputs, reducing the vanishing-gradient problem. It is computationally simple and often accelerates convergence. ReLU also promotes sparsity,

which can improve generalization.

17. What is the difference between a CNN, RNN, and Transformer?

A CNN processes local spatial structure and is widely used for images. An RNN processes sequences step-by-step using recurrent connections and hidden states. A Transformer uses attention mechanisms to model relationships across the entire input sequence in parallel, making it highly scalable.

18. What is a Transformer?

A Transformer is a neural architecture built entirely around self-attention mechanisms rather than recurrence or convolution. It computes relationships between all pairs of input tokens simultaneously. Transformers dominate modern NLP and vision tasks due to their parallelism and long-range modeling ability.